CO1 – ASSIGNMENT 1

Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative

Student: Sudharsan R **Register No.:** 1912260284

Course: C Programming **Submission date:** 27-10-2026

TASK 1 – Understand the Problem and Identify the Data (4 Marks)

1. Inputs, outputs and stored data

Inputs: household/meter number (identifier), previous meter reading, current meter reading, and consumer category if different categories use different tariffs. For a basic implementation, the category is not used because the assignment supplies one tariff. The readings are integers from a 16-bit meter register, 0–65535.

Outputs: units consumed, fixed charge, energy charge for each slab, total bill in paise, and formatted total in rupees. Keeping the bill in paise prevents fractional-paise rounding errors.

Constants: register modulus (65536), fixed charge (5000 paise), free slab limit (100 units), second slab limit (200 units), and the slab rates (0, 225, and 450 paise per unit). These values remain unchanged during one execution and should be defined centrally.

2. Validity rules and assumptions

• Meter readings must be between 0 and 65535 inclusive.

• If current reading is greater than or equal to previous reading, consumption = current – previous.

• If current reading is less than previous reading, assume the 16-bit register rolled over once: consumption = (65536 – previous) + current.

• Consumption, each charge and the final total must never be negative; total bill must equal fixed charge plus all slab charges.

• Assume at most one rollover occurs between readings. If a meter can roll over more than once, timestamps or rollover counters are needed.

• The fixed charge applies even when consumption is zero, as shown by the supplied expected output.

3. Alternative representation and complexity

If readings are stored as text (for example, "01530"), the program must validate that every character is a digit and convert the string to an integer. For a d-digit reading, conversion and validation take O(d) time. The string needs O(d) bytes plus its terminating null character, while a 16-bit integer needs a fixed 2 bytes on the target system. Integer arithmetic is simpler and avoids repeated conversion. The billing calculation itself is O(1) time and O(1) extra space because it performs a fixed number of arithmetic operations.

Evidence expected: the input/output list, assumptions and validity rules above identify what must be stored and checked.

TASK 2 – Choose the Data Types (4 Marks)

1. Recommended data types

Data item	Suggested type	Reason
Meter readings	uint16_t (or unsigned short after checking limits.h)	Exactly 16 bits when uint16_t exists; range 0–65535.
Consumption	unsigned int / uint32_t	Can represent 0–65535 units; wider type also allows future expansion.
Tariff constants and charge	long / int64_t for paise	Integer paise avoids floating-point rounding; use a wide type for totals.

Data item	Suggested type	Reason
Bill output	long long or int64_t	Safe for a large number of units and accumulated bills.
Household/meter number	string or suitable unsigned integer	An identifier is not used for arithmetic; text preserves leading zeros.
Category	enum	Makes named categories clearer and reduces accidental invalid values.

The actual size of C types is implementation-dependent. Use sizeof and limits.h to verify size and range on the compiler/system used. A 16-bit unsigned register has maximum 65535; adding 1 in a 16-bit register wraps to 0. A signed 16-bit short cannot represent all meter readings.

2. Why not float rupees?

Binary floating-point cannot represent many decimal fractions exactly. If thousands of bills are added, small representation errors can accumulate and rounding may occur at different stages. Store money as integer paise, calculate exactly, and convert only when displaying: rupees = paise / 100 and paise remainder = paise % 100.

3. Complete code: 16-bit meter rollover

```
#include <stdio.h>
#include <limits.h>

typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
    /* The assignment specifies a 16-bit meter register. */
    if (current >= previous)
        return (unsigned int)current - (unsigned int)previous;

    /* One rollover: (65535 -> 0), modulus is 65536. */
    return (65536U - (unsigned int)previous) + (unsigned int)current;
}

int main(void)
{
    printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
           sizeof(meter_t), (unsigned int)USHRT_MAX);
    printf("Consumption: %u units\n", consumption(1200, 1530));
    printf("Consumption: %u units\n", consumption(65400, 120));
    return 0;
}
```

Expected output on a system with 16-bit unsigned short:

```
sizeof(meter_t) = 2 bytes, maximum reading = 65535
Consumption: 330 units
Consumption: 256 units
```

The second result is 256 because the register counts 65400 to 65535 (136 counts), wraps to 0, then counts to 120: 136 + 120 = 256. The calculation uses unsigned wider arithmetic, so it does not produce a negative value. If unsigned short is not 16 bits on the compiler, use uint16_t from and verify that it is available.

Alternative concept and complexity

If the register value were stored as a double rupee amount rather than integer paise, each addition could introduce a small binary floating-point error. Integer paise gives exact addition for whole paise values. Both approaches take O(1) time per bill and O(1) space per bill, but integer paise is exact within the chosen integer range.

TASK 3 – Define the Constants (3 Marks)

1. Constant definitions and complete program

```
#include <stdio.h>

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0       /* first 100 units are free */
#define RATE2_PAISE 225     /* Rs 2.25 per unit */
#define RATE3_PAISE 450     /* Rs 4.50 per unit above 200 */
#define FIXED_CHARGE_PAISE 5000L  /* Rs 50.00 */

long long compute_bill(int units)
{
    long long paise = FIXED_CHARGE_PAISE;

    if (units <= SLAB1_UNITS) {
        paise += (long long)units * RATE1_PAISE;
    } else if (units <= SLAB2_UNITS) {
        paise += (long long)(units - SLAB1_UNITS) * RATE2_PAISE;
    } else {
        paise += (long long)(SLAB2_UNITS - SLAB1_UNITS) * RATE2_PAISE;
        paise += (long long)(units - SLAB2_UNITS) * RATE3_PAISE;
    }
    return paise;
}

int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    int n = (int)(sizeof tests / sizeof tests[0]);

    for (int i = 0; i < n; i++) {
        long long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %lld.%02lld\n",
               tests[i], p / 100, p % 100);
    }
    return 0;
}
```

Expected output:

```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
```

2. Why use #define, const and enum?

#define: preprocessor text replacement; it has no variable storage and no C type checking. Use uppercase names for macros. **const:** a typed read-only object that can be inspected in a debugger and may occupy storage. **enum:** best for a set of named integral choices such as consumer categories. For these fixed tariff numbers, macros meet the task requirement; a const-based design can improve type checking and debugging. Changing RATE2_PAISE from 225 to 230 in one definition updates every calculation that uses the second slab. Runtime cost is effectively unchanged.

Alternative concept and complexity

Using const variables instead of #define gives typed values and easier debugging, but may allocate storage depending on the compiler and use. An enum is appropriate for categorical values, not ideal for currency rates requiring a specific type. The slab calculation performs a fixed number of comparisons and arithmetic operations: O(1) time and O(1) extra space.

TASK 4 – Use Macros Safely (4 Marks)

1. Safe macro definitions

```
#include <stdio.h>

#define RUPEES_TO_PAISE(r) ((long long)(r) * 100LL)
#define SQUARE(x)          ((x) * (x))
#define MAX(a, b)          (((a) > (b)) ? (a) : (b))

/* Prefer a function for expressions with side effects. */
static inline int max_int(int a, int b)
```

```
{
    return (a > b) ? a : b;
}

int main(void)
{
    int x = 5;
    int y = 9;

    printf("Rs 12 in paise = %lld\n", RUPEES_TO_PAISE(12));
    printf("Square of %d = %d\n", x, SQUARE(x));
    printf("Maximum = %d\n", MAX(x, y));
    printf("Function maximum = %d\n", max_int(x, y));
    return 0;
}
```

Expected output:

```
Rs 12 in paise = 1200
Square of 5 = 25
Maximum = 9
Function maximum = 9
```

2. Macro pitfalls and how to avoid them

• **Parentheses:** define SQUARE(x) as ((x) * (x)), not x*x. Without parentheses, SQUARE(2+3) may expand to 2+3*2+3 and give 11 instead of 25.

• **Repeated evaluation:** MAX(a,b) may evaluate one argument more than once. MAX(i++, j++) can increment a variable multiple times and is unsafe. Use a function such as max_int when arguments may have side effects.

• **Preprocessor expansion:** gcc -E source.c shows expanded source; gcc -E -P source.c removes line markers. gcc -E is evidence that macros are expanded before compilation.

• **Function versus macro:** an inline function provides type checking, evaluates each argument once, and is easier to debug. A macro may avoid a function-call cost, but modern compilers often inline small functions.

3. Complexity comparison

For a macro used k times, the expanded expression is copied into the program at each use, so generated source/object code may grow with k. A function has one function definition and a call at each use; if it is inlined, the compiler may also duplicate the code. For simple arithmetic, both have O(1) time per use and O(1) working space; the key differences are code size, type safety, debugging and repeated argument evaluation. A preprocessor macro itself has no run-time call overhead, but it is not automatically faster than an optimised inline function.

4. Compilation and validation commands

```
gcc -std=c11 -Wall -Wextra -pedantic task2.c -o task2
./task2
gcc -std=c11 -Wall -Wextra -pedantic task3.c -o task3
./task3
gcc -std=c11 -Wall -Wextra -pedantic task4.c -o task4
./task4
gcc -E -P task4.c > task4_preprocessed.c
```

The code snippets are separate programs; save each in its own file (task2.c, task3.c and task4.c) before compiling. For the rollover code, the displayed sizeof output assumes unsigned short is 2 bytes and has maximum 65535 on the target system.

Conclusion

A reliable billing program should validate readings, explicitly handle one meter rollover, store money as integer paise, centralise tariff constants, and use fully parenthesised macros only where appropriate. The supplied test cases verify ordinary consumption, rollover, tariff slabs and macro calculations.

CO1 – ASSIGNMENT 2

Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative

Student: Sudharsan R **Register No.:** 1912260284

Course: C Programming **Submission date:** 27-10-2026

TASK 1 – Organise the Program with Header Files and Conditional Compilation (3 Marks)

Program structure and header file

A well-organised C program separates declarations and shared constants into a header file, while the implementation and main() remain in .c files. Include guards prevent the same header from being processed more than once. Use **#include <header.h>** for system headers found through compiler include paths and **#include "header.h"** for a project header searched relative to the source file first.

File: billing.h

```
#ifndef BILLING_H
#define BILLING_H

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0LL
#define RATE2_PAISE 225LL
#define RATE3_PAISE 450LL
#define FIXED_CHARGE_PAISE 5000LL

long long compute_bill(int units);

#ifdef DEBUG
#include <stdio.h>
#define DEBUG_PRINT(...) \
    do { fprintf(stderr, "[DEBUG] "); \
         fprintf(stderr, __VA_ARGS__); \
         fputc('\n', stderr); } while (0)
#else
#define DEBUG_PRINT(...) ((void)0)
#endif

#endif /* BILLING_H */
```

File: billing.c

```
#include "billing.h"

long long compute_bill(int units)
{
    long long paise = FIXED_CHARGE_PAISE;

    if (units <= SLAB1_UNITS) {
        paise += (long long)units * RATE1_PAISE;
    } else if (units <= SLAB2_UNITS) {
        paise += (long long)(units - SLAB1_UNITS) * RATE2_PAISE;
    } else {
        paise += (long long)(SLAB2_UNITS - SLAB1_UNITS) * RATE2_PAISE;
        paise += (long long)(units - SLAB2_UNITS) * RATE3_PAISE;
    }
    return paise;
}
```

File: main.c

```
#include <stdio.h>
#include "billing.h"

int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    int count = (int)(sizeof tests / sizeof tests[0]);

    for (int i = 0; i < count; ++i) {
        long long bill = compute_bill(tests[i]);
        DEBUG_PRINT("units=%d, bill=%lld paise", tests[i], bill);
        printf("%d units -> Rs %lld.%02lld\n",
               tests[i], bill / 100, bill % 100);
    }
    return 0;
}
```

Compile and run:

```
gcc -std=c11 -Wall -Wextra -pedantic main.c billing.c -o billing
./billing
gcc -std=c11 -Wall -Wextra -pedantic -DDEBUG main.c billing.c -o billing_debug
./billing_debug
```

Normal build prints bills only. The DEBUG build additionally prints intermediate units and bill values to standard error. Conditional compilation removes the debug calls from the normal build, so there is no run-time check for DEBUG in every call.

Alternative and complexity

If debugging used a run-time variable such as if (debug) printf(...), each reached debug point performs a condition check, even when debugging is off. The extra time is O(1) per debug point and total overhead is O(k) for k points; the executable retains the debug strings and code. With #ifdef DEBUG, those statements are absent from a non-debug executable, reducing code size and removing the run-time branch. Include guards also prevent duplicate declarations and definitions caused by repeated inclusion.

TASK 2 – Read and Print Data Correctly (4 Marks)

Correct format specifiers and input validation

Use the format specifier that matches the argument type: %d for int, %u for unsigned int, %ld for long, %lld for long long, %f for double in printf, %lf for double in scanf, %c for char, and %s for a character array. A mismatched format specifier causes undefined behaviour, which may print incorrect values or corrupt data. For scanf, check the return value: it must equal the number of conversions expected.

```
#include <stdio.h>

int main(void)
{
    int units;
    long long bill_paise;

    printf("Enter units consumed: ");
    if (scanf("%d", &units) != 1 || units < 0) {
        fprintf(stderr, "Invalid units. Enter a non-negative integer.\n");
        return 1;
    }

    printf("Enter bill in paise: ");
    if (scanf("%lld", &bill_paise) != 1 || bill_paise < 0) {
        fprintf(stderr, "Invalid bill amount.\n");
        return 1;
    }

    printf("Units: %d\n", units);
    printf("Bill: Rs %lld.%02lld\n",
           bill_paise / 100, bill_paise % 100);
    return 0;
}
```

Example input: 150 units and 16250 paise. Output: Units: 150; Bill: Rs 162.50. The validation detects letters or missing values because scanf returns fewer conversions than requested. In production, fgets() plus strtol() is more robust for rejecting trailing junk and detecting range errors.

Alternative: fgets() and strtol()

Reading a line with fgets() allows the program to inspect the entire input before conversion. For a line of length L, reading and validating it takes O(L) time and the buffer uses O(L) space. scanf() is concise and works for simple controlled input, but recovery from malformed input can be harder; fgets() with strtol() makes it easier to detect trailing characters, empty input and out-of-range values. A fixed buffer should be large enough and the program should detect whether the line was truncated.

TASK 3 – Control Type Conversion and Precision (4 Marks)

Where conversions occur

C promotes small integer types such as char and short to int in most arithmetic expressions. When signed and unsigned values are combined, the usual arithmetic conversions can convert a negative signed value to a large unsigned value. Integer division discards the fractional part. Cast before multiplication when a wide result is needed, and use integer paise for money so every charge is exact to one paise.

```
#include <stdio.h>

int main(void)
{
    int units = 150;
    int paise_per_unit = 225;
    int whole_rupees = 16250 / 100;
    long long exact_paise = (long long)units * paise_per_unit;
    double rupees = exact_paise / 100.0;
```

```
    printf("Surprise 1: 5 / 2 = %d (integer division)\n", 5 / 2);
    printf("Surprise 2: (double)(5 / 2) = %.1f\n",
           (double)(5 / 2));
    printf("Surprise 3: (double)5 / 2 = %.1f\n",
           (double)5 / 2);
    printf("150 * 225 paise = %lld paise = Rs %.2f\n",
           exact_paise, rupees);
    printf("16250 / 100 as integer = %d rupees\n", whole_rupees);
    return 0;
}
```

Expected output:

```
Surprise 1: 5 / 2 = 2 (integer division)
Surprise 2: (double)(5 / 2) = 2.0
Surprise 3: (double)5 / 2 = 2.5
150 * 225 paise = 33750 paise = Rs 337.50
16250 / 100 as integer = 162 rupees
```

The three expressions differ because casting after integer division cannot restore the discarded fraction, while casting an operand before division makes the operation floating-point. The final line intentionally shows that integer division loses the 50 paise; use bill_paise / 100 and bill_paise % 100 when formatting a bill.

Casting and rounding rule

For large totals, cast before multiplication: (long long)units * rate_paise. Casting after a potentially overflowing int multiplication is too late. If a tariff calculation genuinely creates a fractional paise, use a documented rule such as round-half-up at the final conversion, rather than rounding each intermediate step. With integer paise slab rates, no fractional-paise rounding is needed.

Alternative and complexity

Integer-paise arithmetic takes O(1) time and O(1) space per bill and has no binary floating-point rounding error, provided the integer type does not overflow. Double arithmetic also takes O(1) time and O(1) space per bill, but values such as 0.1 are not represented exactly in binary. If n bills are accumulated in floating point, error can accumulate with n; the exact bound depends on magnitudes and operation order. Integer paise is preferred for this billing problem.

TASK 4 – Trace the Compilation Process (4 Marks)

Compilation stages and commands

The C build process has four main stages: preprocessing, compilation to assembly, assembling to object code, and linking object files and libraries into an executable.

```
gcc -E main.c > main.i       # preprocessed C
gcc -S main.c -o main.s     # assembly text
gcc -c main.c -o main.o     # object file
gcc -c billing.c -o billing.o
gcc main.o billing.o -o billing  # link executable
./billing
```

Preprocessing expands macros, includes header contents, and handles conditional compilation such as #ifdef DEBUG. Compilation translates C into assembly and checks syntax and types. Assembling creates object code. Linking resolves references such as compute_bill() across main.o and billing.o and adds required libraries.

Classifying errors

Error type	Example	When it appears / solution
Compile-time	Missing semicolon or undeclared variable	Compiler reports syntax/name error; fix source and rebuild.
Link-time	Call compute_bill() but omit billing.o	Linker reports undefined reference; compile and link billing.c too.
Run-time	Invalid input, division by zero, or out-of-bounds access	Program runs but fails or behaves incorrectly; validate inputs and test.

Warnings and how to resolve them

Compile with gcc -std=c11 -Wall -Wextra -Wconversion -pedantic. Warnings can identify suspicious conversions, unused variables, or format mismatches. Read each warning, confirm whether it reveals a real bug, then correct the type, add a justified cast, validate the input, or remove unused code. Do not silence warnings without understanding them.

Alternative: separate source files

With one source file, changing any part usually requires recompiling the entire file. With several .c files and a shared header, only changed source files need recompilation, then the object files are linked again. For a small project, the time saved is modest; for large projects it can be substantial. Separate object files use additional disk space, but each object contains only compiled code and data for its source file. If a shared header changes, every source file depending on it should be recompiled; a build tool such as make tracks these dependencies.

Suggested README evidence

Include your name and register number, compiler version (gcc --version), the exact commands above, screenshots or copied output from each task, and a short note explaining what each result verifies. Create one folder per task and commit progress

Conclusion

The program is easier to maintain when shared declarations are placed in guarded headers, debug logging is controlled at compile time, input is validated, money is stored in paise, conversions are explicit, and compilation stages are tested separately. These practices improve correctness, portability and debugging.

CO1 – ASSIGNMENT 3

Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative

Student: Sudharsan R **Register No.:** 1912260284

Course: C Programming **Submission date:** 27-10-2026

TASK 1 – Implement and Test the Billing Program (4 Marks)

Complete C program

```
#include <stdio.h>
#include <limits.h>

#define MODULUS 65536U
#define SLAB1_UNITS 100U
#define SLAB2_UNITS 200U
#define RATE1_PAISE 0LL
#define RATE2_PAISE 225LL
#define RATE3_PAISE 450LL
#define FIXED_CHARGE_PAISE 5000LL

typedef unsigned short meter_t;

int meter_consumption(unsigned int previous, unsigned int current,
                      unsigned int *units)
{
    if (previous > 65535U || current > 65535U || units == NULL)
        return 0;
    if (current >= previous)
        *units = current - previous;
    else
        *units = (MODULUS - previous) + current; /* one rollover */
    return 1;
}

long long compute_bill(unsigned int units)
{
    long long bill = FIXED_CHARGE_PAISE;
    if (units <= SLAB1_UNITS)
        bill += (long long)units * RATE1_PAISE;
    else if (units <= SLAB2_UNITS)
        bill += (long long)(units - SLAB1_UNITS) * RATE2_PAISE;
    else {
        bill += (long long)(SLAB2_UNITS - SLAB1_UNITS) * RATE2_PAISE;
        bill += (long long)(units - SLAB2_UNITS) * RATE3_PAISE;
    }
    return bill;
}

void print_bill(unsigned int units)
{
    long long p = compute_bill(units);
    printf("%u units -> Rs %lld.%02lld\n", units, p / 100, p % 100);
}

int main(void)
{
    unsigned int tests[] = {0, 99, 100, 101, 150, 199, 200, 201, 350};
    unsigned int n = sizeof tests / sizeof tests[0];

    printf("Meter type size: %zu bytes; USHRT_MAX=%u\n",
           sizeof(meter_t), (unsigned int)USHRT_MAX);
    for (unsigned int i = 0; i < n; ++i)
        print_bill(tests[i]);

    unsigned int units;
    if (meter_consumption(65400U, 120U, &units))
        printf("Rollover consumption: %u units\n", units);

    if (!meter_consumption(70000U, 120U, &units))
        printf("Invalid meter reading rejected\n");

    return 0;
}
```

Expected output on a system where unsigned short is 2 bytes and maximum 65535:

```
Meter type size: 2 bytes; USHRT_MAX=65535
0 units -> Rs 50.00
99 units -> Rs 50.00
100 units -> Rs 50.00
101 units -> Rs 52.25
150 units -> Rs 162.50
199 units -> Rs 272.75
200 units -> Rs 275.00
201 units -> Rs 279.50
350 units -> Rs 950.00
Rollover consumption: 256 units
Invalid meter reading rejected
```

Test interpretation

The boundary tests show the first 100 units are free, units 101–200 cost ■2.25 each, and units above 200 cost ■4.50 each. A 16-bit meter reading that falls from 65400 to 120 is treated as one rollover: (65536 − 65400) + 120 = 256 units. Readings outside 0–65535 are rejected. In a real application, a timestamp or rollover counter should be used if more than one rollover could occur between readings.

Array/loop alternative and complexity

Tariff limits and rates could be stored in arrays and processed by a loop. If there are s tariff slabs, the loop takes O(s) time and O(s) space for the tariff arrays. The current implementation uses a fixed number of conditions, so billing is O(1) time and O(1) extra space. A loop is easier to extend when tariffs become numerous or configurable; fixed branches are simple for only three slabs.

TASK 2 – Compare Expectations with Execution (4 Marks)

1. Float versus integer-paise totals

The integer-paise version stores every charge as an exact integer. Floating-point values such as ■2.25 are commonly represented exactly or closely on binary systems depending on the value, but decimal fractions such as ■0.10 are not generally exact. A floating-point total can therefore differ slightly from an integer-paise total. For the given tariff, the per-unit rates are whole paise, so integer paise is the most reliable method.

2. Sample total for scale comparison

Households/bills	Example average bill	Total	Fits signed 32-bit int paise?
4,000	■500	■20,00,000 = 200,000,000 paise	Yes
60,000	■500	■3,00,00,000 = 3,000,000,000 paise	No (exceeds 2,147,483,647)
800,000	■500	■40,00,00,000 = 40,000,000,000 paise	No

These are illustrative totals assuming ■500 average bill, not measured bills. They show why a 32-bit signed integer is insufficient for district-wide totals in paise. A 64-bit signed integer is recommended. An int total in rupees might fit these examples but cannot represent paise exactly; use a wide integer paise total.

3. Type sizes and limits

C only guarantees minimum ranges for standard integer types; exact sizes vary by implementation. Use sizeof and limits.h on the target compiler. Common systems use 4-byte int and 8-byte long long, but the report must use the actual measurements from the lab system. A 64-bit signed integer has a maximum of 9,223,372,036,854,775,807, comfortably above the illustrative 40 billion paise total. Floating-point overflow is less likely at this scale, but precision loss can occur before overflow.

Alternative: double district total

A double accumulator typically uses 8 bytes and has a large range, but it cannot exactly represent every decimal fraction. At 800,000 bills, small per-bill errors may accumulate; the exact error depends on input values and summation order. A long long accumulator also typically uses 8 bytes, is exact for integer paise, and costs O(1) time per addition. Both approaches use O(1) space per accumulator; integer paise is preferred for correctness.

TASK 3 – Recommend and Defend the Final Design (4 Marks)

Recommended design

• Meter readings: use a verified 16-bit unsigned type (uint16_t where available) for the meter register and a wider unsigned integer for consumption calculations.

• Money: store tariff rates and bills as integer paise; use long long or int64_t for totals.

• Tariffs: centralise fixed charge, slab limits and rates in one header or configuration module so a revision is made once.

• Validation: reject invalid readings, negative units, malformed input and impossible register values; handle rollover explicitly.

• Testing: run zero, slab boundaries, rollover, invalid-input and hand-calculated bill cases; retain test evidence.

• Debugging: compile debug output conditionally with #ifdef DEBUG, and compile normal builds without it.

Limitations and tariff changes

The basic model assumes one rollover at most between readings and a single tariff for all consumers. Smart meters sending readings every 15 minutes make multiple rollovers between readings unlikely, but timestamps and meter IDs should still be checked. A time-of-day tariff requires separate consumption totals by time band; simply applying one rate to all units would be wrong. If the tariff changes, update the central configuration or load a validated configuration file.

Configuration file alternative

Reading tariff values from a configuration file lets authorised staff change rates without recompiling. It adds start-up parsing time O(s) for s tariff entries and O(s) storage for those entries. Recompiling constants has no start-up parsing cost but requires rebuilding and redistributing the program after every tariff revision. For a co-operative that expects frequent tariff changes, a validated configuration file is preferable, with versioning, audit logs and error handling.

DAY 4 DESIGN CHALLENGE – TASK 4: Re-evaluate and Adapt the Design (8 Marks)

Changed requirements

The new smart meters report watt-hours every 15 minutes, approximately 35,040 readings per household per year. Units consumed must be computed from cumulative energy readings or interval readings, and a time-of-day tariff adds a 20% surcharge for energy used from 6 PM to 10 PM. Tariff values must be changeable without recompiling.

Decision: MODIFY the original design

Keep integer-paise accounting, explicit validation and modular functions, but modify the data model and tariff source. Store meter energy readings in a sufficiently wide integer type, such as uint64_t if the meter specification supports it. If values are cumulative, compute the difference between readings and handle meter rollover using the meter's documented modulus. If readings are interval watt-hours, validate each interval and aggregate into the correct time band. Store tariff configuration in a file read at start-up, and validate all limits and rates before processing bills.

Time-of-day calculation

For each interval, determine the time band from its timestamp. For intervals fully within 6 PM–10 PM, apply the 20% surcharge; for intervals outside it, use the base tariff. If an interval crosses 6 PM or 10 PM, split the interval at the boundary or obtain finer-grained meter data so energy is allocated correctly. Do not infer the whole interval's tariff from only its end time.

Suggested data model

```
typedef struct {
    unsigned long long energy_wh;
    unsigned int tariff_category;
    /* timestamp and meter ID should also be stored */
} meter_reading_t;

/* Store currency as integer paise; use a wide integer total.
   Load base rates and slab limits from a validated config file.
   Use integer arithmetic and a documented rounding rule for any
   20% surcharge that produces a fractional paise. */
```

For a 20% surcharge, calculate in a wider integer numerator and divide only once with a documented rounding rule, for example (base_paise * 120 + 50) / 100 for non-negative values using round-half-up. Ensure the multiplication cannot overflow; validate the rate and use a sufficiently wide intermediate type.

Complexity, reliability and limitations

For n readings and s tariff slabs, a direct fixed-slab calculation is O(n) overall; a loop over tariff slabs is O(n·s). With only a few slabs, both are efficient. Configuration parsing takes O(s) time and O(s) space once at program start. A single reading can be processed in O(1) space, while storing a year's full readings requires O(r) space for r readings per household. The design must also handle missing/out-of-order readings, clock changes, daylight-saving rules where relevant, duplicate records, meter resets, configuration errors and audit requirements.

Conclusion

The recommended solution retains integer-paise billing and explicit validation, while modifying the design for higher-resolution smart-meter data and configurable tariffs. Wide integer types support district-scale totals, and timestamp-aware allocation ensures the evening surcharge is applied only to eligible energy. Test results should be recorded from the actual compiler and system before final submission.